

RELEX

This repository is the official implementation of the CHI 2023 Paper entitled ["RELEX: Scaling Example-Based Learning via Natural Language Processing"]

Usage guide

A. Run Locally

A1. Set-up

1. The code was developed with Python 3.8.8. We recommend creating a python virtual environment (to avoid conflicts with other packages). Use virutalenv, pyenv or conda env to do that.

```
conda create --name relex-env python=3.8.8
conda activate relex-env
```

2. Clone the code

```
git clone <<link-to-repository>>
cd <<name-of-respository>>
```

3. Install dependencies

```
pip install -r requirements.txt
```

A2. Database

The database is used to store the examples and log users' clickstream activity.

1. Create a Postgres database. You can follow [this tutorial](#). There are multiple ways to create a Postgres database, we recommend downloading [PgAdmin](#) and to create the new database by clicking on Servers>PostgreSQL>Databases>Create>Database. Name the database "relex".

- Alternatively, you can create the database in Heroku (see Host online section).
2. By default, your database should have the following parameters. If something is different, update the parameters in backend/recipe/postgres_utils.py

```
user = "postgres"
password = ""
port = 5432
database_name = "relex"
hostname = "localhost"
```

3. Download the examples dataset from [here](#) and place in database/data/
4. Run the script start_database.py to create the schemas and tables for the metadata and to load the examples.

```
python ../../database/start_database.py
```

A3. Run app

1. To run the backend, go to the backend folder and run the backend using uvicorn

```
cd backend
uvicorn recipe.app:app --reload
```

- You may test the APIs at <http://127.0.0.1:8000/docs>
2. To run the frontend, open frontend/client/recipe-1.html file with any browser.

B. Host online

We recommend using Heroku to host the backend and frontend. Note that Heroku has free credits for students.

B1. Back-end

1. In Heroku, create a new app called "relex-backend".
2. In "relex-backend" add a Heroku Postgres database as an installed add-on.
3. Go to Settings>Config Vars to obtain the key of the database. Uncomment line 29 in backend/recipe/postgres_utils.py and change the DATABASE URL to match the key of the database.

```
DATABASE_URL = os.environ['HEROKU_POSTGRESQL_TEAL_URL']
```

4. Push the backend code (folder backend) to GitHub and connect the repo to Heroku: Deploy>Deploy method: GitHub. Then, select the repo in "Apps connected to GitHub".

B2. Front-end

1. In Heroku, create a new app called "relex-frontend".
2. Push the frontend code (folder frontend) to GitHub and connect the repo to Heroku: Deploy>Deploy method: GitHub. Then, select the repo in "Apps connected to GitHub".
3. Click on "Open App" and you will see the application running.

C. Run in custom domain

C1. Fine-tune model

1. Upload the examples in the model/data/raw_example.csv. The file must have two columns: "text" and "evaluation".
2. Run the fine-tuning pipeline. Pass as a parameter the name of the model

```
python ./model/model_pipeline.py --name chemistry
```

3. Add the model name in line 15 of backend/recipe/nlp.py.

C2. Interface

1. Replace the suggestions in the folder with the following format:

component	text	explanation	first	second	exception	error	color	missing
ingredients	Indicated whether cheese should be shredded, crumbled, cubed or another form.		cheese	shredded crumbled cubed grated cubes sliced diced slice	cream cheese Philadelphia soup cake dressing topping macaroni		2	1 2 1

Where the columns have the following meanings: - component: whether the rule applies to one part of ingredients/material or to the whole text. - text: the suggestion that the users will see. - explanation: extra information about the suggestion that will be provided when the user hovers over the suggestion. - first: keywords to look for where the rule applies. - second: in case there is a second word that should appear - exception: words that make the suggestion not apply even if the first keywords are present. - error: if the rule is suggestion is on things that should not be done, the keywords should be written here - color: color code for the interface - missing: the value to assign if the suggestion is broken. By default is a 1 (does not apply), but in some situations, you might want to have a 0 (not present).

2. Upload the new examples to the database. Run the two following commands:

```
python ./database/format_examples.py
python ./database/start_database.py
```

3. Upload the text from the suggestions in the interface. First, run the file update_rules.py. Copy the output from the terminal and replace lines 179-410 in frontend/client/recipe-1.html.

```
python ./transfer/update_rules.py
```

4. Upload the colors in the interface. Analogously, run the file `update_colors.py`. Copy the output from the terminal and replace lines 13-690 in `frontend/client/css/style.css`.

```
python ./transfer/update_colors.py
```

5. Run the application locally or host it as indicated in previous sections.

Example datasets and suggestions to custom the domain

Domain	Text	Evaluation	Suggestions	Example Dataset
Expository writing	Article	Reliability score of the articles (e.g., sources, neutral point of view, no contradictions)	Reliability Criteria	Wikipedia Articles with reliability scores
Procedural writing	Chemistry lab reports	School grade from teacher	Handbook on Writing Laboratory Reports	Sample lab reports
Procedural writing	Code documentation (readme.md)	GitHub stars	How to write a good readme	Github repositories per domain with stars
Help-seeking requests	Forum questions	Quality of post (regarding scores, number of community edits, and community acceptance)	How to ask good questions guide	Stack Overflow questions with quality of posts
Speech	Discourse	Popularity (likes)	Speech Writing	Ted Talks with likes
Law	Legal cases	Case outcomes, whether the case was referred to, followed, considered, applied, discussed, distinguished, approved	Best practices for law cases	Australian legal cases with outcomes from the Federal Court of Australia (FCA)

Contributing

This code is provided for educational purposes and aims to facilitate the reproduction of our design and further research in this direction. We have done our best to document, refactor, and test the code before publication.

If you find any bugs or would like to contribute new features, feel free to file issues and pull requests on the repo and we will address them as we can.

License

This code is free software: you can redistribute it and/or modify it under the terms of the [MIT License](#).

This software is distributed in the hope that it will be useful, but without any warranty; without even the implied warranty of merchantability or fitness for a particular purpose. See the [MIT License](#) for details.